CampusConnect — Student Welfare Management System

DBMS Course Project · Mid-Sem Database Design Document · RDBMS: PostgreSQL 13+ · Team: Group-15 – Shubham Sagar (2024A3PS0394H), Jay Kimtani (2024A3PS0392H), Sai Gautam Reddy (2024A3PS0429H), Krishna Gupta (2024A3PS0380H), Sreejith Gundla (2024A3PS0427H)

1. Problem and users

Student-welfare services on a residential campus (modelled on the BITS SWD portal) – hostel rooms, campus stores and food outlets, outpasses, document verification and the academic calendar – are usually run through separate registers and portals, so the same facts are kept in several places and drift apart. CampusConnect keeps them in **one relational database** whose rules live in the DBMS: a student can hold only one active room, an order can only contain items of the store it was placed with, an outpass cannot return before it leaves, and order totals and a student's cumulative spend are maintained by triggers rather than by application code.

User	What they do
Student	Browse store menus and place / cancel orders; view own room and allocation history; raise outpass requests and record return; upload documents and see verification status; view the campus calendar.
Admin (warden / SWD staff)	Maintain hostels, rooms and allocations; approve or reject outpass requests; verify or reject documents; manage stores and menus; publish holidays, events and exams.
Analyst / management	Reports: spend per store and student, popular items, hostel occupancy, outpass approval rate and overdue returns.

2. Main functionality

- **Hostel and room allocation** – allocate, transfer and vacate a student; full stay history is kept; room occupancy is updated automatically by a trigger.
- **Campus store ordering** – a student places a multi-item order in **one transaction**; item prices are snapshotted; totals and the student's running spend are derived by triggers; cancel / reinstate adjust the spend.
- **Outpass management** – request → admin review (approve / reject) → return recorded; overdue and pending queues.
- **Documents** – student uploads (metadata + file path); admin sets PENDING / VERIFIED / REJECTED.
- **Calendar** – holidays, events and exams as an independent entity; searchable by date range and type.
- **Reports and integrity demo** – 5+ non-trivial SQL reports (joins, grouping, window functions, CTEs, subqueries) and an Integrity Lab showing invalid operations rejected by named constraints / triggers.

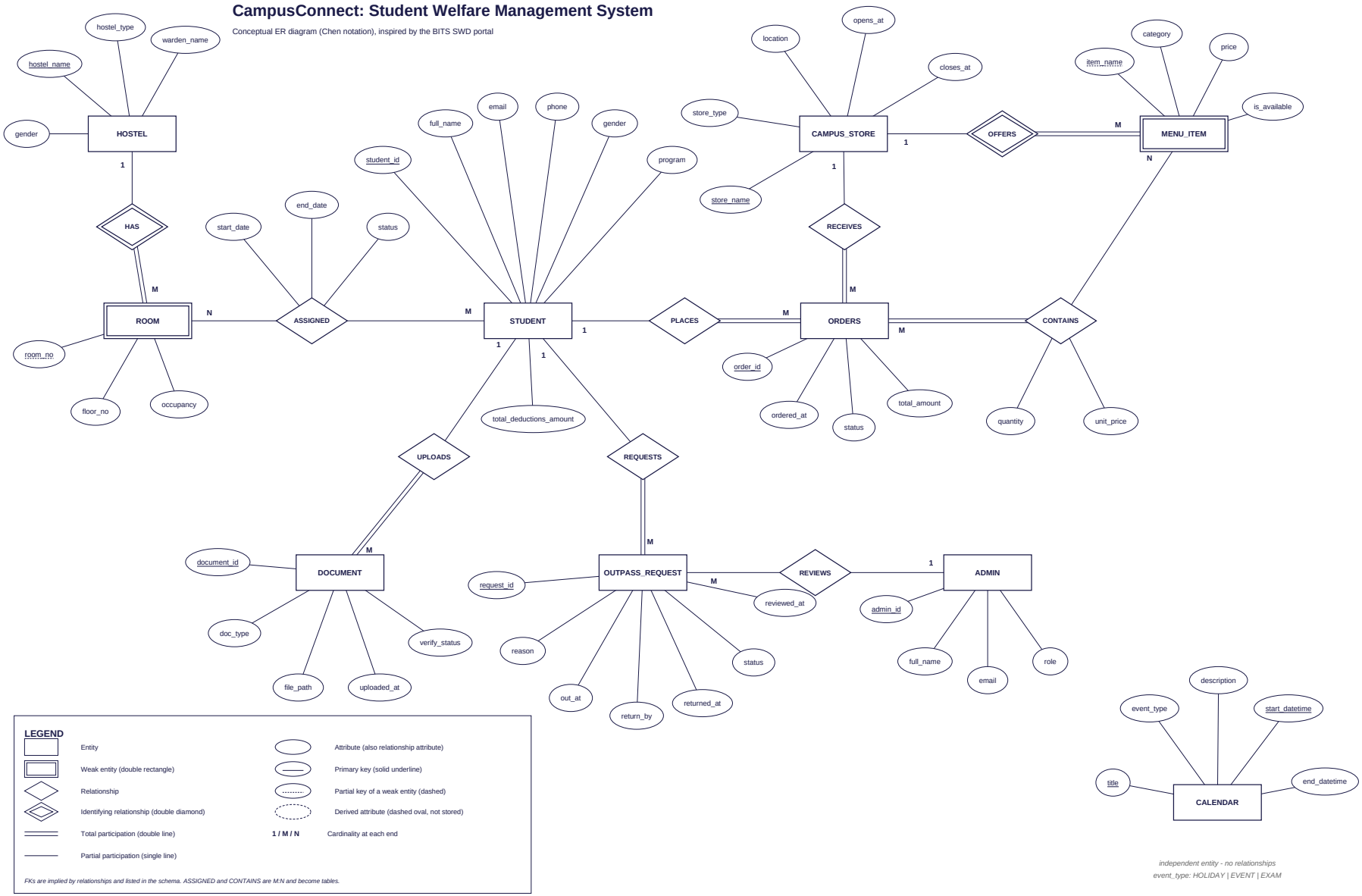
3. Assumptions

- Authentication is an application-layer concern and is not modelled; students are identified by their institute ID (`student_id`), admins by a surrogate key.
- A student has **at most one ACTIVE room allocation** at a time; past stays are retained (history). Room capacity is not modelled – `occupancy` is only a counter of active allocations.
- `hostel_name` and `store_name` are unique campus-wide and used as natural keys. HOSTEL.gender describes the residents housed; it is not forced to equal STUDENT.gender (can be added as a rule).
- Prices are in INR with two decimals. Payments / wallets are **out of scope**; “deductions” means cumulative spend on non-cancelled orders.
- A cancelled order keeps its own total but no longer counts towards the student's spend; its items are frozen. Menu price changes never alter past orders (unit_price snapshot).
- Files are stored outside the database; DOCUMENT holds only the path and verification metadata. CALENDAR holds holidays, events and exams only (no academic deadlines) and is not linked to ADMIN.

4. Business rules (and where they are enforced)

Rule	Enforced by
One ACTIVE allocation per student; ACTIVE if and only if end_date IS NULL; end_date ≥ start_date	Partial unique index uq_one_active_allocation; CHECK chk_alloc_status, chk_alloc_dates
ROOM.occupancy equals the number of ACTIVE allocations of that room	Trigger trg_alloc_occupancy (fn_alloc_occupancy)
An order can contain only items offered by the store that received it	Composite FKs: ORDER_ITEM(order_id, store_name) → ORDERS(order_id, store_name) and (store_name, item_name) → MENU_ITEM
An order must contain at least one item (total participation in CONTAINS)	Deferred constraint triggers ctrg_order_has_items, ctrg_item_delete_keeps_one (checked at COMMIT)
ORDERS.total_amount = Σ quantity × unit_price; STUDENT.total_deductions_amount = Σ total of non-cancelled orders	Triggers trg_item, trg_order (delta updates); audit queries must return zero rows
Items of a CANCELLED order cannot be inserted, changed or deleted	Trigger trg_item_frozen
Outpass: return_by > out_at; returned_at ≥ out_at; reviewed_by and reviewed_at are set together or not at all	CHECK chk_outpass_window, chk_outpass_returned, chk_outpass_reviewed
Valid domains: gender, hostel gender, order / outpass / allocation / document status, event_type, price ≥ 0, quantity > 0	CHECK constraints; NOT NULL and UNIQUE (student.email, admin.email)

5. ER / EER diagram



Notation (Chen). Rectangles: entities (PK underlined, dashed underline = partial key). Double rectangles: weak entities ROOM (owner HOSTEL via HAS) and MENU_ITEM (owner CAMPUS_STORE via OFFERS); double diamonds: identifying relationships. Double lines: total participation. 1 / M / N: cardinality at each end. **Constructs used:** (i) weak entities; (ii) two M:N relationships with attributes – ASSIGNED (STUDENT–ROOM: start_date, end_date, status) and CONTAINS (ORDERS–MENU_ITEM: quantity, unit_price) – mapped to ROOM_ALLOCATION and ORDER_ITEM; (iii) 1:N relationships PLACES, RECEIVES, UPLOADS, REQUESTS, REVIEWS; (iv) an independent entity CALENDAR. Stored derived values (ORDERS.total_amount, STUDENT.total_deductions_amount, ROOM.occupancy) are trigger-maintained.

6. Relational schema (12 relations)

PK = primary key, FK = foreign key, UQ = unique. All columns are NOT NULL unless marked (null). Surrogate keys use `GENERATED ALWAYS AS IDENTITY`. Full DDL: `db/schema.sql`.

Relation	Attributes	Keys / foreign keys / constraints
HOSTEL	hostel_name, hostel_type, warden_name (null), gender	PK hostel_name; CHECK gender IN (MALE, FEMALE, COED)
ROOM *(weak, HAS)*	hostel_name, room_no, floor_no, occupancy	PK (hostel_name, room_no); FK hostel_name → HOSTEL ON DELETE CASCADE; CHECK floor_no ≥ 0, occupancy ≥ 0
STUDENT	student_id, full_name, email, phone (null), gender (null), program (null), total_deductions_amount	PK student_id; UQ email; CHECK gender IN (MALE, FEMALE, OTHER); total_deductions_amount NUMERIC(12,2) DEFAULT 0 CHECK ≥ 0 (trigger-maintained)
ROOM_ALLOCATION *(M:N ASSIGNED)*	student_id, hostel_name, room_no, start_date, end_date (null), status	PK (student_id, hostel_name, room_no, start_date); FK student_id → STUDENT; FK (hostel_name, room_no) → ROOM; partial UQ (student_id) WHERE status = ACTIVE; CHECK chk_alloc_dates, chk_alloc_status
CAMPUS_STORE	store_name, store_type (null), location (null), opens_at (null), closes_at (null)	PK store_name
MENU_ITEM *(weak, OFFERS)*	store_name, item_name, category (null), price, is_available	PK (store_name, item_name); FK store_name → CAMPUS_STORE ON DELETE CASCADE; CHECK price ≥ 0; is_available DEFAULT TRUE
ORDERS	order_id, student_id, store_name, ordered_at, status, total_amount	PK order_id; UQ (order_id, store_name) – target of composite FK; FK student_id → STUDENT (PLACES); FK store_name → CAMPUS_STORE (RECEIVES); CHECK status IN (PLACED, PREPARING, READY, COMPLETED, CANCELLED), total_amount ≥ 0
ORDER_ITEM *(M:N CONTAINS)*	order_id, store_name, item_name, quantity, unit_price	PK (order_id, item_name); FK (order_id, store_name) → ORDERS (CASCADE); FK (store_name, item_name) → MENU_ITEM; CHECK quantity > 0, unit_price ≥ 0
DOCUMENT	document_id, student_id, doc_type, file_path, uploaded_at, verify_status	PK document_id; FK student_id → STUDENT ON DELETE CASCADE (UPLOADS); CHECK verify_status IN (PENDING, VERIFIED, REJECTED)
ADMIN	admin_id, full_name, email, role (null)	PK admin_id; UQ email
OUTPASS_REQUEST	request_id, student_id, reviewed_by (null), reason, out_at, return_by, returned_at (null), status, reviewed_at (null)	PK request_id; FK student_id → STUDENT (REQUESTS); FK reviewed_by → ADMIN ON DELETE SET NULL (REVIEWS); CHECK status IN (PENDING, APPROVED, REJECTED, CANCELLED); chk_outpass_window / returned / reviewed
CALENDAR	title, event_type, description (null), start_datetime, end_datetime (null)	PK (title, start_datetime); CHECK event_type IN (HOLIDAY, EVENT, EXAM); CHECK end_datetime IS NULL OR ≥ start_datetime; no FKs in or out

7. ER → relational mapping decisions

- **Strong entities** → one relation each. Short, stable names are used as natural keys (hostel_name, store_name, student_id); ORDERS, DOCUMENT, OUTPASS_REQUEST and ADMIN use surrogate identity keys; CALENDAR uses the natural composite (title, start_datetime).
- **Weak entities** ROOM and MENU_ITEM take the owner's key plus the partial key as PK, with ON DELETE CASCADE from the owner.
- **M:N ASSIGNED** → ROOM_ALLOCATION. start_date is added to the PK so a student can return to the same room later and history is kept; the “one active room” rule is a partial unique index.
- **M:N CONTAINS** → ORDER_ITEM with quantity and unit_price. store_name is carried into ORDER_ITEM so two composite FKs force every item of an order to come from the store that received it – a declarative rule instead of a trigger.
- **1:N relationships** (PLACES, RECEIVES, UPLOADS, REQUESTS, REVIEWS) → FK on the N side. REVIEWS is partial (new requests are unreviewed), so reviewed_by is a nullable FK with ON DELETE SET NULL; all other FK columns are NOT NULL (total participation).

8. Integrity constraints

Kind	Examples (constraint : rule)
Keys	12 primary keys (5 composite: room, room_allocation, menu_item, order_item, calendar); 3 UNIQUE constraints (student.email, admin.email, uq_order_store) + partial unique index uq_one_active_allocation; 11 foreign keys (3 composite: room_allocation → room, order_item → orders, order_item → menu_item).
Domain CHECK	Enumerated values for hostel / student gender, allocation / order / outpass / document status and event_type; floor_no ≥ 0; occupancy ≥ 0; price ≥ 0; quantity > 0; unit_price ≥ 0; total_amount ≥ 0; total_deductions_amount ≥ 0.
Multi-column CHECK	chk_alloc_dates (end ≥ start); chk_alloc_status ((status = ACTIVE) = (end_date IS NULL)); chk_outpass_window (return_by > out_at); chk_outpass_returned; chk_outpass_reviewed ((reviewed_by IS NULL) = (reviewed_at IS NULL)); chk_calendar_dates.
Referential actions	CASCADE only along ownership (HOSTEL → ROOM, CAMPUS_STORE → MENU_ITEM, ORDERS → ORDER_ITEM, STUDENT → DOCUMENT); SET NULL for ADMIN → reviewed_by; all other references are restrictive so orders, outpasses and stay history cannot disappear by accident.
Triggers (cross-row rules)	trg_item, trg_order, trg_item_frozen, trg_alloc_occupancy, and deferred constraint triggers ctrg_order_has_items / ctrg_item_delete_keeps_one (error raised at COMMIT if an order has no item).

9. DBMS features used / planned

Feature	Objects	Purpose	Status
Triggers (6 triggers, 5 functions)	trg_item, trg_order, trg_item_frozen, ctrg_order_has_items, ctrg_item_delete_keeps_one, trg_alloc_occupancy	Maintain derived totals and occupancy; freeze cancelled orders; enforce “≥ 1 item” at commit	Implemented
Indexes (10)	uq_one_active_allocation (partial unique) + 9 FK / filter indexes: idx_alloc_room, idx_orders_student, idx_orders_store, idx_item_menu, idx_document_student, idx_outpass_student, idx_outpass_reviewer, idx_outpass_status, idx_calendar_start	Indexes on FK columns (PostgreSQL does not create them automatically) and on the outpass status / calendar date filters	Implemented
Transactions	Place order (order + items), transfer student (end old stay + insert new), review outpass	Order rows and items commit together so the deferred “≥ 1 item” check and the trigger-maintained totals stay consistent; delta updates (column = column + x) serialise concurrent orders without lost updates	Schema ready; wired in app for final
Views / stored functions	v_student_spend, v_hostel_occupancy, v_overdue_outpass; sp_place_order (JSONB items)	Reporting and one-call order placement	Planned (final)
Analytic SQL	JOINs, GROUP BY / HAVING, RANK() window, CTEs, correlated subquery, double NOT EXISTS	≥ 5 reports (see Application Design)	Planned (final)

10. Normalization

1NF: all attributes are atomic; repeating groups are separate relations (order lines → ORDER_ITEM, stays → ROOM_ALLOCATION). **2NF:** in every composite-key relation each non-key attribute depends on the whole key – ROOM(hostel_name, room_no) → floor_no, occupancy; MENU_ITEM(store_name, item_name) → category, price, is_available; ROOM_ALLOCATION(student_id, hostel_name, room_no, start_date) → end_date, status; ORDER_ITEM(order_id, item_name) → quantity, unit_price. **3NF / BCNF:** HOSTEL(hostel_name) → hostel_type, warden_name, gender; the only other determinants are candidate keys (student_id, email; admin_id, email; order_id). No non-key attribute determines another non-key attribute, so the relations are in BCNF apart from the deliberate exceptions below.

11. Controlled redundancy (deliberate exceptions)

- **ORDERS.total_amount** is determined by ORDER_ITEM rows. Kept so order lists and the spend total need no join; written only by trg_item, never by the application. Audit query: totals must equal Σ quantity × unit_price.
- **STUDENT.total_deductions_amount** is determined by non-cancelled ORDERS. Kept for O(1) spend look-ups; maintained by trg_order using atomic delta updates; audited against the orders table.
- **ORDER_ITEM.store_name** is determined by order_id (via ORDERS). Kept on purpose so composite FKs enforce “items belong to the order's store”; it cannot drift because it is part of an FK to ORDERS.
- **ROOM.occupancy** and **ROOM_ALLOCATION.status** are derivable (count of ACTIVE rows; end_date IS NULL). Occupancy is trigger-maintained; the CHECK ties status to end_date. **ORDER_ITEM.unit_price** is a deliberate price snapshot, not redundancy.

12. Other design decisions

- **Rules in the database, not the app:** the web layer does no business-rule validation; it passes PostgreSQL's SQLSTATE (23505 unique, 23503 foreign key, 23514 check, P0001 raised by triggers) back with a readable message.
- **History is never cascaded away:** students, rooms, stores and menu items are referenced restrictively, so deleting them while orders, allocations or outpasses exist fails instead of erasing history.
- **Optional hardening:** REVOKE UPDATE on the trigger-maintained columns (total_amount, total_deductions_amount, occupancy) from the application role; trigger functions then run SECURITY DEFINER.
- **Trade-off accepted:** natural keys (hostel_name, store_name) make child keys wider but keep joins and reports readable; surrogate keys are used where no stable natural key exists (orders, documents, outpasses, admins).

13. Application architecture (summary – details in Application_Design.pdf)

Browser single-page app (React + Vite) ↔ REST API (Node.js + Express) ↔ PostgreSQL through `node-postgres` (connection pool, parameterised queries only). Three end-to-end workflows: **(1) Place / cancel an order** – one transaction; triggers update totals and the student's spend; **(2) Outpass lifecycle and reports** – request → review → return, overdue and approval-rate reports (joins, aggregates, CTEs); **(3) Room allocation and Integrity Lab** – transfer a student atomically; invalid operations (second active room, item from another store, edit of a cancelled order, outpass returning before leaving) are rejected by named constraints / triggers.

14. Progress at mid-sem and team contributions

Item	Status
Requirements, ER diagram (Chen), relational mapping, normalization	Done
PostgreSQL schema: 12 tables, keys, CHECKs, 10 indexes, 6 triggers, audit queries (db/schema.sql)	Done
Seed data generator (≥ 500 records), views, stored function, 5+ queries	In progress → final
Backend REST API and React front end for the 3 workflows	Planned → final

Member (name, ID)	Contribution up to mid-sem
Shubham Sagar (2024A3PS0394H)	Requirements analysis, problem statement, assumptions and business rules
Jay Kimtani (2024A3PS0392H)	ER diagram (Chen notation) and ER-to-relational mapping
Sai Gautam Reddy (2024A3PS0429H)	Relational schema DDL: keys, data types, CHECK / UNIQUE constraints
Krishna Gupta (2024A3PS0380H)	Triggers, indexes and audit queries
Sreejith Gundla (2024A3PS0427H)	Normalization and design justification, application design, README, GitHub repo